

CS 352

Computer Graphics

Lecture 2

Graphics Softwares and Filling Algorithms

Computer Graphics Software

- Two broad categories:
 - Special purpose packages (e.g. CAD systems and painting programs)
 - General programming packages (e.g. Graphics Library (GL), OpenGL, Virtual Reality Modeling Language (VRML), Java2D, Java3D)

Fundamental Graphic Functions

There are fundamental graphic functions which are essentially to be supported by computer system:

- 1 The ability to establish a graphic screen mode: EGA, CGA, VGA, and SVGA.
- 2 The ability to clear the screen.
- 3 The ability to draw graphic primitives: Line, circle, ellipse, rectangle, ... etc.
- 4 The ability to fill areas with colors.
- 5 The ability to put an alphanumeric (text) material onto the graphic screen.
- 6 The ability to return back to the default text mode.

Graphics Software Standards

- **Graphics work stations from Silicon Graphics Inc. (SGI):** became increasingly popular with the development of GKS and PHIGS
- **Graphics Library (GL):** set of routines came with the graphics work stations
- **OpenGL:** developed in the early 1990s as a HW independent version of GL

OpenGL Vs. WebGL

- **OpenGL** is a desktop computer-centric API (like Direct3D). The following table lists a set of technologies related to OpenGL.
- **WebGL** is derived from OpenGL ES 2.0 (intended for mobile devices) which has less capabilities and is simpler to use. WebGL is also designed to run in a browser, and has therefore a few limitations more than OpenGL ES 2.0.

API	Technology Used
OpenGL ES	It is the library for 2D and 3D graphics on embedded systems - including consoles, phones, appliances, and vehicles. OpenGL ES 3.1 is its latest version. It is maintained by the Khronos Group www.khronos.org
JOGL	It is the Java binding for OpenGL. JOGL 4.5 is its latest version and it is maintained by jogamp.org .
WebGL	It is the JavaScript binding for OpenGL. WebGL 1.0 is its latest version and it is maintained by the khronos group .
OpenGLSL	OpenGL Shading Language. It is a programming language which is a companion to OpenGL 2.0 and higher. It is a part of the core OpenGL 4.4 specification. It is an API specifically tailored for embedded systems such as those present on mobile phones and tablets.

OpenGL Vs. WebGL

❖ Who Developed WebGL

An American-Serbian software engineer named **Vladimir Vukicevic** did the foundation work and led the creation of WebGL

- In 2007, Vladimir started working on an **OpenGL** prototype for Canvas element of the HTML document.
- In March 2011, Kronos Group created WebGL.

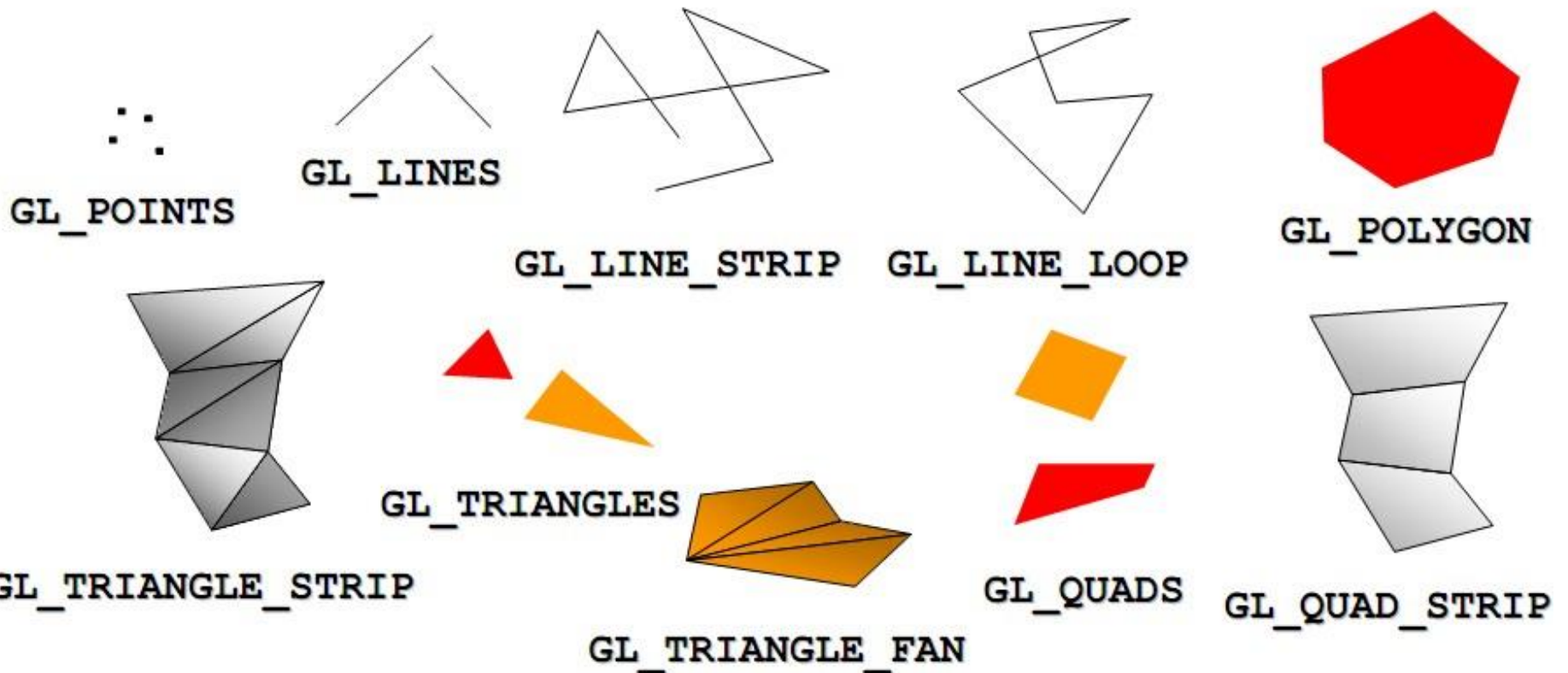
❖ Advantages of WebGL

Here are the advantages of using WebGL –

- **JavaScript programming** – WebGL applications are written in JavaScript. Using these applications, you can directly interact with other elements of the HTML Document. You can also use other JavaScript libraries (e.g. JQuery) and HTML technologies to enrich the WebGL application.
- **Increasing support with mobile browsers** – WebGL also supports Mobile browsers such as iOS safari, Android Browser, and Chrome for Android.
- **Open source** – WebGL is an open source. You can access the source code of the library and understand how it works and how it was developed.
- **No need for compilation** – JavaScript is a half-programming and half-HTML component. To execute this script, there is no need to compile the file. Instead, you can directly open the file using any of the browsers and check the result. Since WebGL applications are developed using JavaScript, there is no need to compile WebGL applications as well.

Geometric Primitives in OpenGL

- All geometric primitives are specified by vertices
- Depending on how vertices are organized, OpenGL can render any of the shown primitives.



Geometric Primitives in OpenGL

The inputs to the Graphics Rendering Pipeline are geometric primitives (such as triangle, point, line or quad), which is formed by one or more vertices.

OpenGL interprets the vertices and renders each primitive using the following rules:

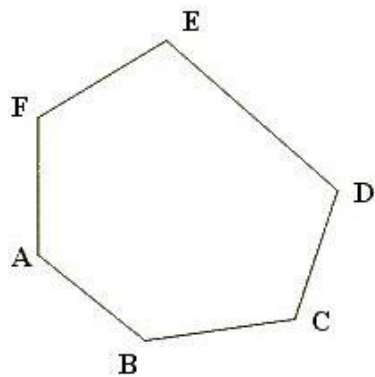
- **GL_LINES**—To draw unconnected line segments. **OpenGL** draws a line segment for each group of two vertices. If the application specifies **n vertices**, **OpenGL renders $n/2$ line segments**. If n is odd, OpenGL ignores the final vertex.
- **GL_LINE_STRIP**— To draw a sequence of connected line segments. OpenGL renders a line segment between the first and second vertices, between the second and third and so on. If the application specifies **n vertices**, OpenGL renders $n-1$ line segments.
- **GL_LINE_LOOP**— Use this primitive to close a line strip. OpenGL renders this primitive like a `GL_LINE_STRIP` with the addition of a closing line segment between the final and first vertices.
- **GL_TRIANGLES**— Use this primitive to draw individual triangles. **OpenGL renders a triangle** for each group of **three vertices**. If your application specifies **n vertices**, OpenGL **renders $n/3$ triangles**. If n isn't a multiple of 3, OpenGL ignores the excess vertices.

Geometric Primitives in OpenGL

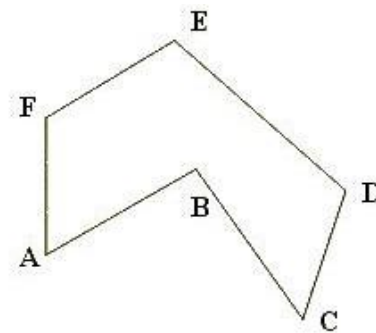
- **GL_TRIANGLE_STRIP**— Use this primitive to draw a sequence of triangles that share edges. OpenGL renders a triangle using the first, second, and third vertices, and then another using the second, third, and fourth vertices, and so on. If the application specifies n vertices, OpenGL renders $n-2$ connected triangles. If n is less than 3, OpenGL renders nothing.
- **GL_TRIANGLE_FAN**— Use this primitive to draw a fan of triangles that share edges and also share a vertex. Each triangle shares the first vertex specified. If the application specifies a sequence of vertices v , OpenGL renders a triangle using v_0 , v_1 , and v_2 ; another triangle using v_0 , v_2 , and v_3 ; another triangle using v_0 , v_3 , and v_4 ; and so on. If the application specifies n vertices, OpenGL renders $n-2$ connected triangles. If n is less than 3, OpenGL renders nothing.
- **GL_QUADS**— Use this primitive to draw. OpenGL renders a quadrilateral for each group of individual convex quadrilaterals four vertices. If the application specifies n vertices, OpenGL renders $n/4$ quadrilaterals. If n isn't a multiple of 4, OpenGL ignores the excess vertices.
- **GL_QUAD_STRIP**— Use this primitive to draw a sequence of quadrilaterals that share edges. If the application specifies a sequence of vertices v , OpenGL renders a quadrilateral using v_0 , v_1 , v_3 , and v_2 ; another quadrilateral using v_2 , v_3 , v_5 , and v_4 ; and so on. If the application specifies n vertices, OpenGL renders $(n-2)/2$ quadrilaterals. If n is less than 4, OpenGL renders nothing.



POLYGON



(a) Convex

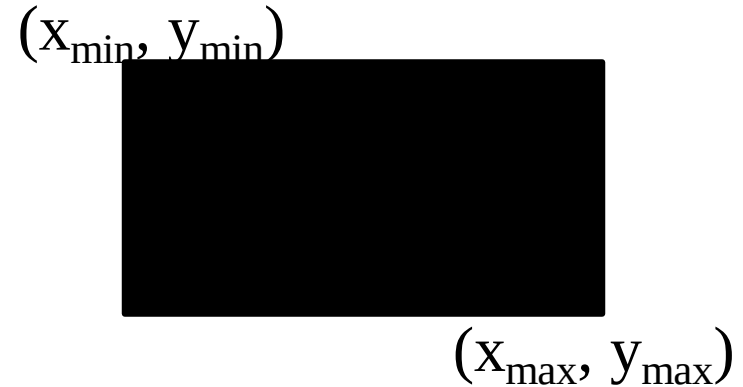


(b) Not convex

Figure 1: Closed polygons

Filling

- **Region Filling**
 - It is a process of coloring in a definite area or region. A region may be described as pixels or geometric level.
- **Filling rectangle:**
 - For ($y = y_{\min}$ to $y_{\max}$ of rectangle)
 - For ($x = x_{\min}$ to $x_{\max}$ of rectangle)
 - WritePixel(x, y)





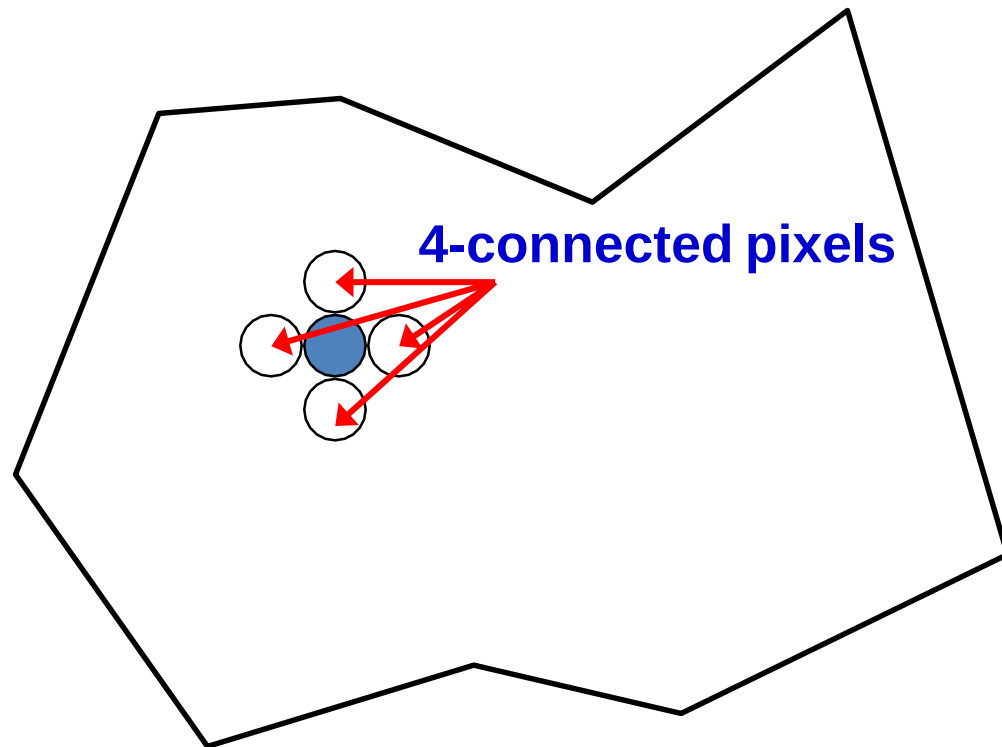
POLYGON FILLING

Polygon is an ordered list of vertices as shown in the above figure. For filling polygons with particular colors, you need to determine the pixels falling on the border of polygon and those which fall inside the polygon.

Now, we will see how we can fill polygons using different techniques.

Basic Filling Algorithm

The basic filling algorithm is commonly used in interactive graphics packages, where the user specifies an interior point of the region to be filled.





Basic Filling Algorithm

- 1 Set the user specified point.
- 2 Store the four neighboring pixels in a stack.
- 3 Remove a pixel from the stack.
- 4 If the pixel is not set,
 Set the pixel
 Push its four neighboring pixels into the stack
- 5 Go to step 3
- 6 Repeat till the stack is empty.

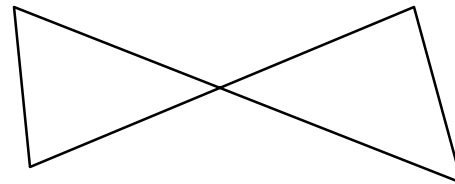
Basic Filling Algorithm (Code)

```
void fill(int x, int y) {  
    if(getPixel(x,y)==0){  
        setPixel(x,y);  
        fill(x+1,y);  
        fill(x-1,y);  
        fill(x,y+1);  
        fill(x,y-1);  
    }  
}
```



Basic Filling Algorithm

- Requires an interior point.
- Involves considerable amount of stack operations.
- The boundary has to be closed.
- Not suitable for self-intersecting polygons





Types of Basic Filling Algorithms

- A) Boundary Fill Algorithm
 - For filling a region with a single boundary color.
 - Condition for setting pixels:
 - Color is not the same as border color
 - Color is not the same as fill color
- B) Flood Fill Algorithm
 - For filling a region with multiple boundary colors.
 - Condition for setting pixels:
 - Color is same as the old interior color

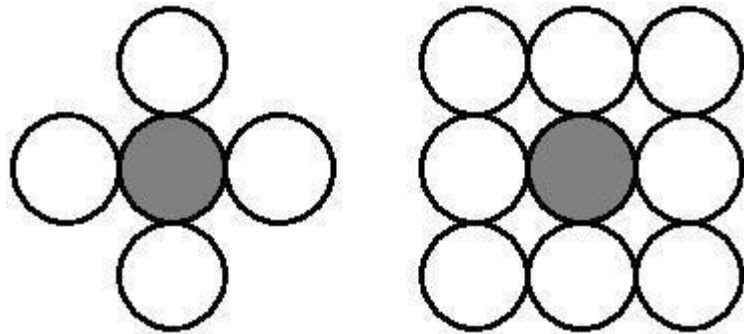
A) Boundary Fill Algorithm

The following steps illustrate the idea of the **recursive** boundary-fill algorithm:

1. Start from an interior point.
2. If the current pixel is **not already** filled and if it is not an edge point, then set the pixel with the fill color, and store its neighboring pixels (**4** or **8-connected**) in the stack for processing. Store only neighboring pixel that is **not already** filled and is not an edge point.
3. Select the next pixel from the stack, and continue with step 2.

A) Boundary Fill Algorithm

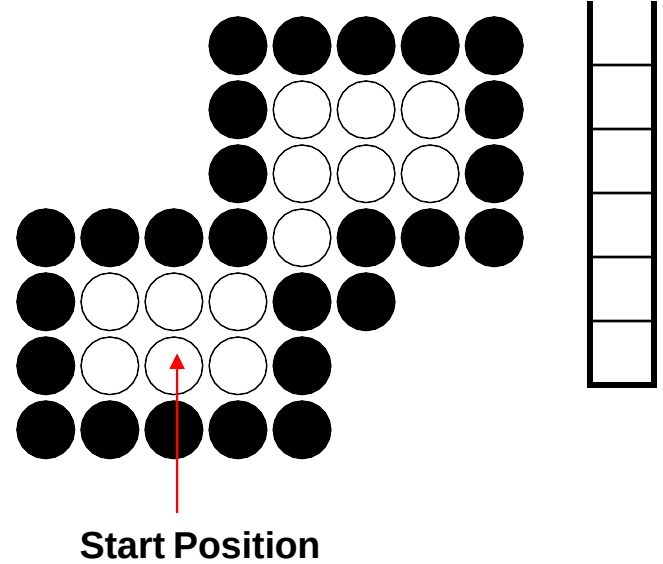
The order of pixels that should be added to stack using **4-connected** is above, below, left, and right. For **8-connected** is above, below, left, right, above-left, above-right, below-left, and below-right.

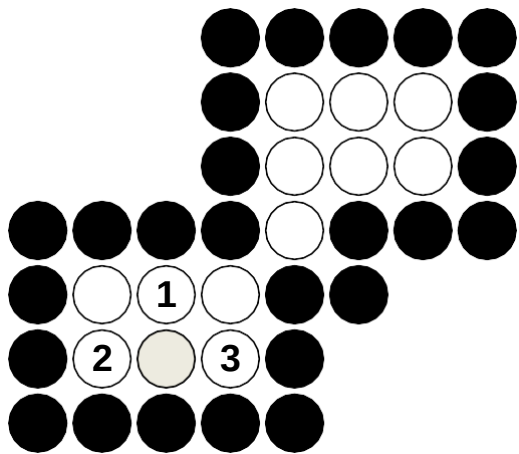


Boundary Fill Algorithm (4-connected)

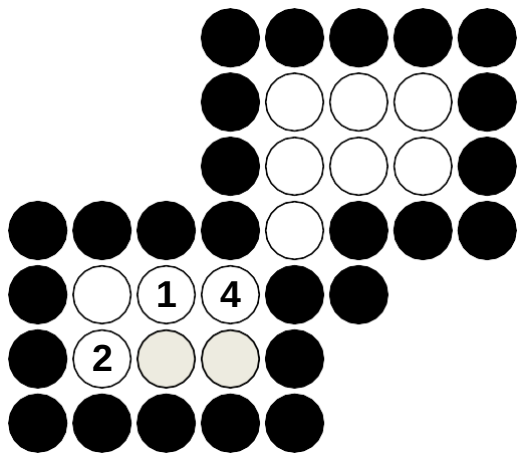
```
void boundaryFill(int x, int y,  
                 int fillColor, int borderColor)  
{  
    getPixel(x, y, color);  
    if ((color != borderColor)  
        && (color != fillColor)) {  
        setPixel(x,y);  
        boundaryFill(x+1,y,fillColor,borderColor);  
        boundaryFill(x-1,y,fillColor,borderColor);  
        boundaryFill(x,y+1,fillColor,borderColor);  
        boundaryFill(x,y-1,fillColor,borderColor);  
    }  
}
```

4-connected (Example)

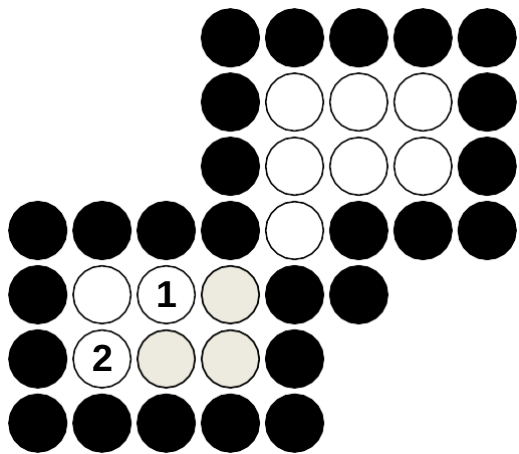




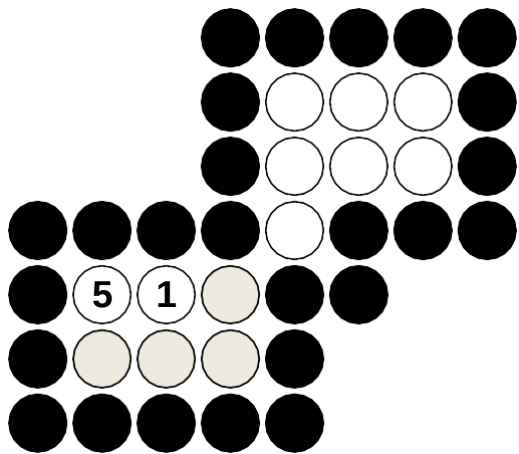
3
2
1



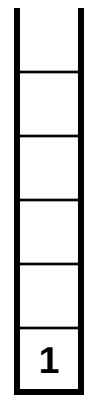
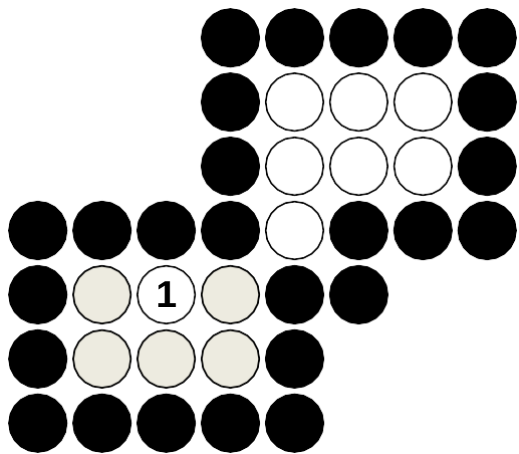
4
2
1

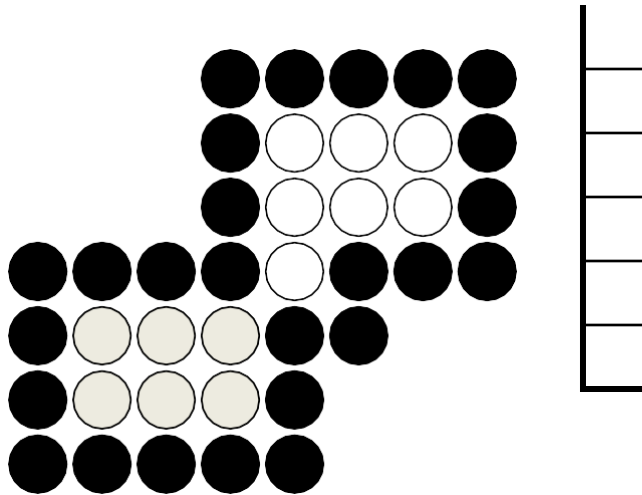


2
1



5
1





Some region remains unfilled

Boundary Fill Algorithm (8-connected)

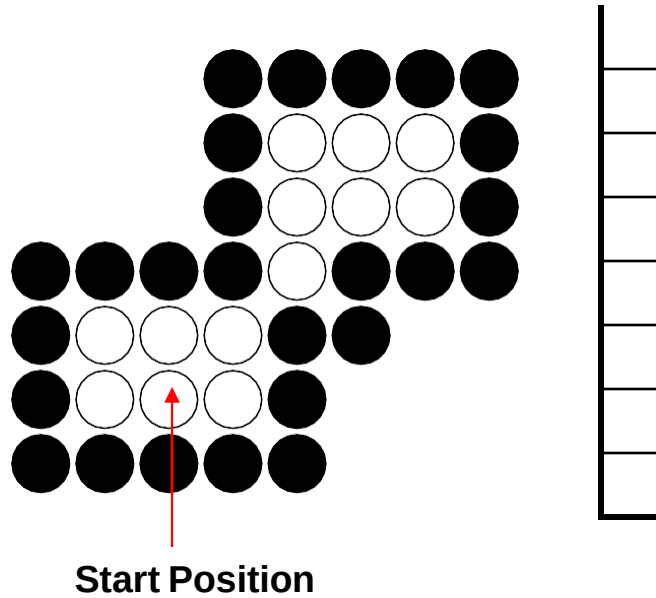
```
void boundaryFill(int x, int y,  
                  int fillColor, int borderColor)  
{  
    getPixel(x, y, color);  
    if ((color != borderColor)  
        && (color != fillColor)) {  
        setPixel(x,y);  
        boundaryFill(x+1,y,fillColor,borderColor);  
        boundaryFill(x-1,y,fillColor,borderColor);  
        boundaryFill(x,y+1,fillColor,borderColor);  
        boundaryFill(x,y-1,fillColor,borderColor);  
        boundaryFill(x+1,y+1,fillColor,borderColor);  
        boundaryFill(x+1,y-1,fillColor,borderColor);  
        boundaryFill(x-1,y+1,fillColor,borderColor);  
        boundaryFill(x-1,y-1,fillColor,borderColor);  
    }
```

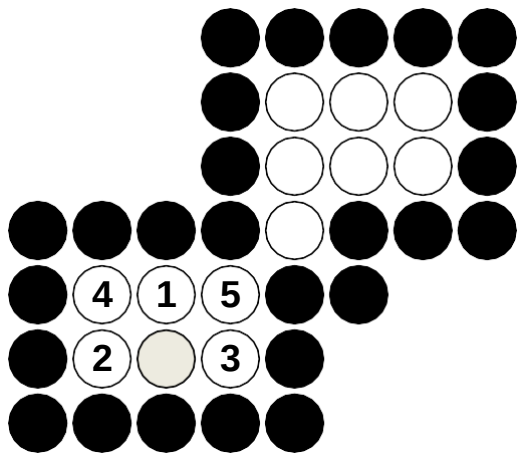


Boundary Fill – 8 connected

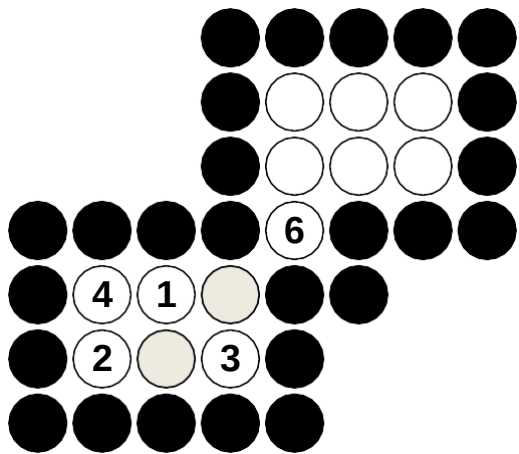
```
flood_fill(int x, int y, int fill_color, int old_color) {  
    if(getpixel(x,y) == old_color) {  
        putpixel(x,y,fill_color);  
        flood_fill(x-1, y , fill_color, old_color);  
        flood_fill(x+1, y , fill_color, old_color);  
        flood_fill(x, y+1 , fill_color, old_color);  
        flood_fill(x, y-1 , fill_color, old_color);  
        flood_fill(x+1, y+1 , fill_color, old_color);  
        flood_fill(x+1, y-1 , fill_color, old_color);  
        flood_fill(x-1, y+1 , fill_color, old_color);  
        flood_fill(x-1, y-1 , fill_color, old_color);  
    }  
}
```

8-connected (Example)

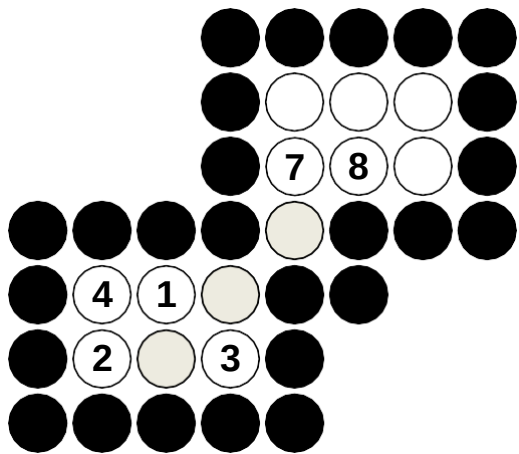




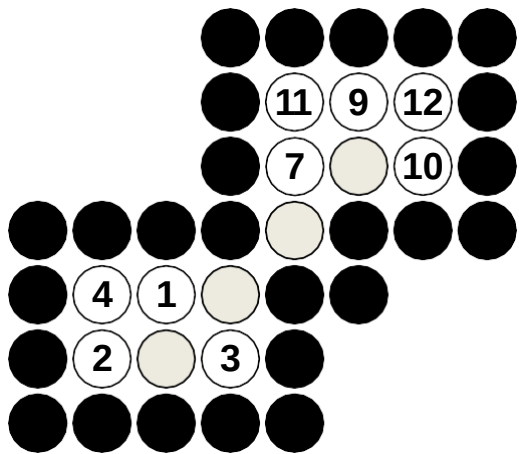
5
4
3
2
1



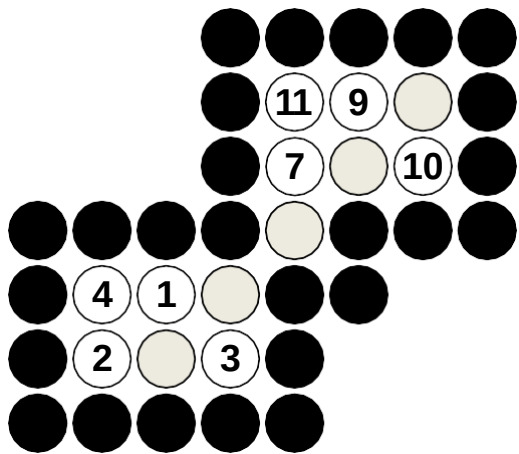
6
4
3
2
1



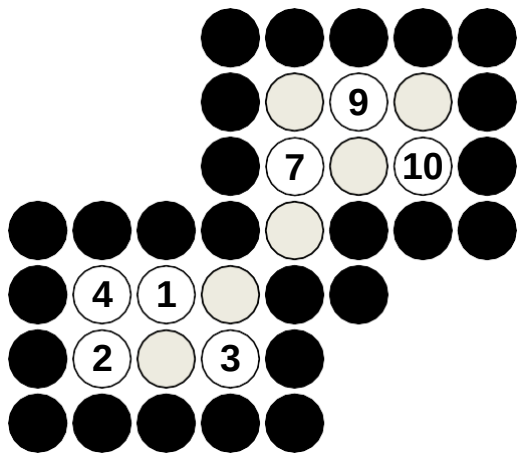
8
7
4
3
2
1



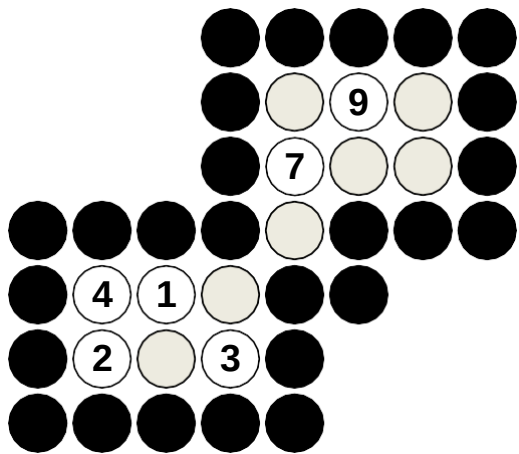
12
11
10
9
7
4
3
2
1



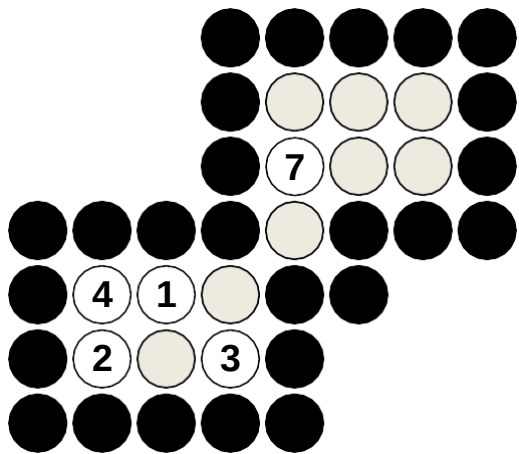
11
10
9
7
4
3
2
1



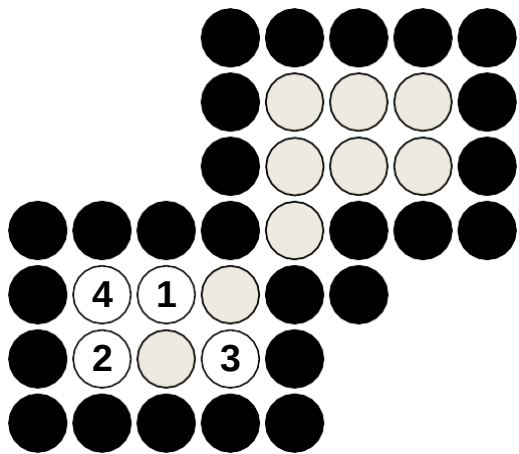
10
9
7
4
3
2
1



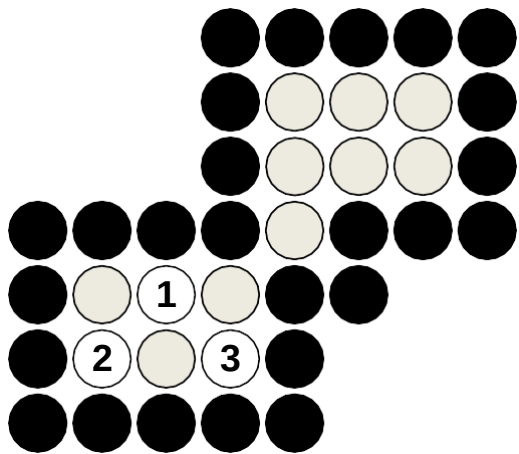
9
7
4
3
2
1



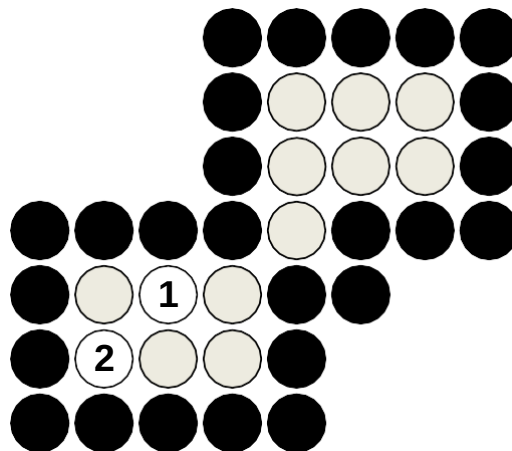
7
4
3
2
1

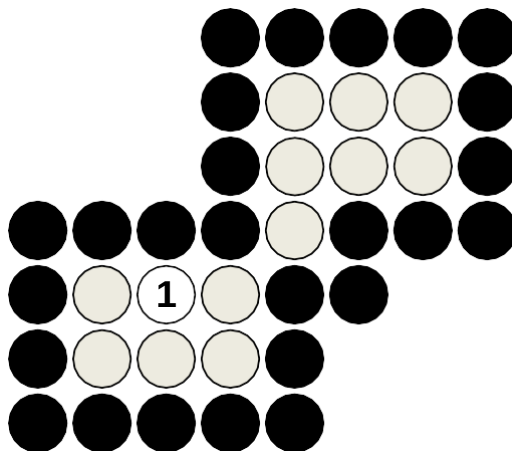


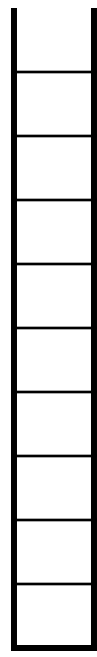
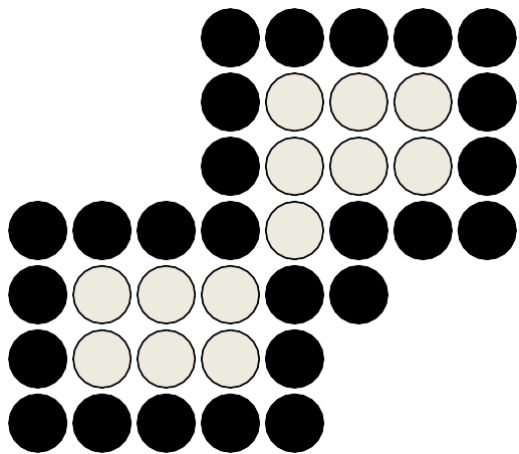
4
3
2
1



3
2
1

[illegible]

[illegible]

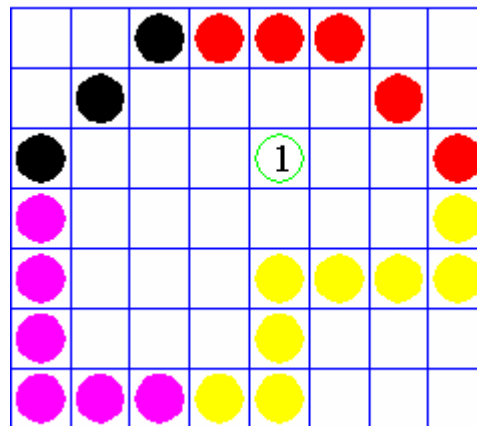


B- Flood Fill Algorithm

Sometimes we want to fill in (recolor) an area that is not defined within a single color boundary.

We paint such areas by replacing a specified interior color instead of searching for a boundary color value.

This approach is called a **flood-fill algorithm**.



B- Flood Fill Algorithm

We start from a specified interior pixel (x, y) and reassign all pixel values that are currently set to a given interior color with the desired fill color.

If the area has **more than one** interior color, we can first **reassign pixel values** so that all interior pixels have the same color.

Using either **4-connected** or **8-connected** approach, we then step through pixel positions until all interior pixels have been repainted.

B) Flood Fill Algorithm (Code)

```
void floodFill(int x, int y,  
               int fillColor, int oldColor)  
{  
    getPixel(x, y, color);  
    if (color != oldColor)  
    {  
        setPixel(x,y);  
        floodFill(x+1, y, fillColor, oldColor);  
        floodFill(x-1, y, fillColor, oldColor);  
        floodFill(x, y+1, fillColor, oldColor);  
        floodFill(x, y-1, fillColor, oldColor);  
    }  
}
```

Flood Fill Algorithm

These methods (**Span Flood-Fill**) fill **horizontal pixel spans across scan lines**, instead of proceeding to 4-connected or 8-connected neighboring pixels.

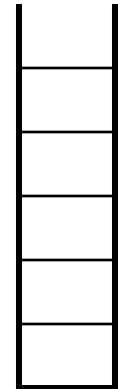
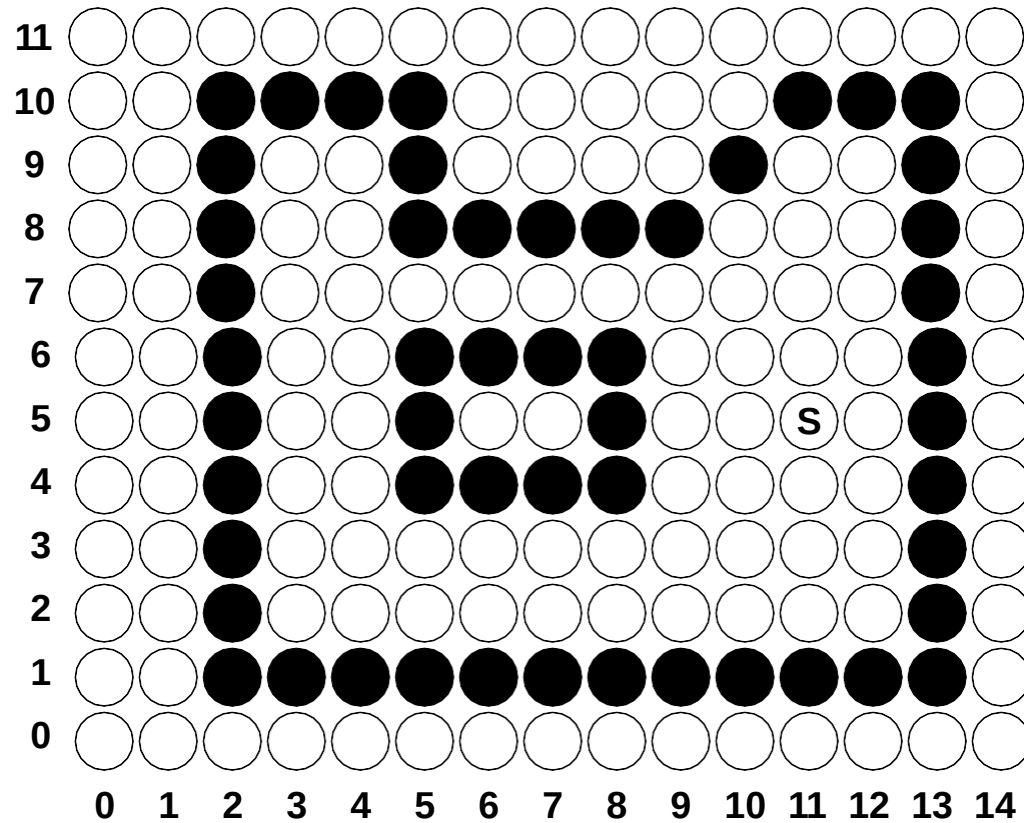
Then we need only stack a beginning position for each horizontal pixel spans, instead of stacking all unprocessed neighboring positions around the current position.

Span Flood-Fill Algorithm

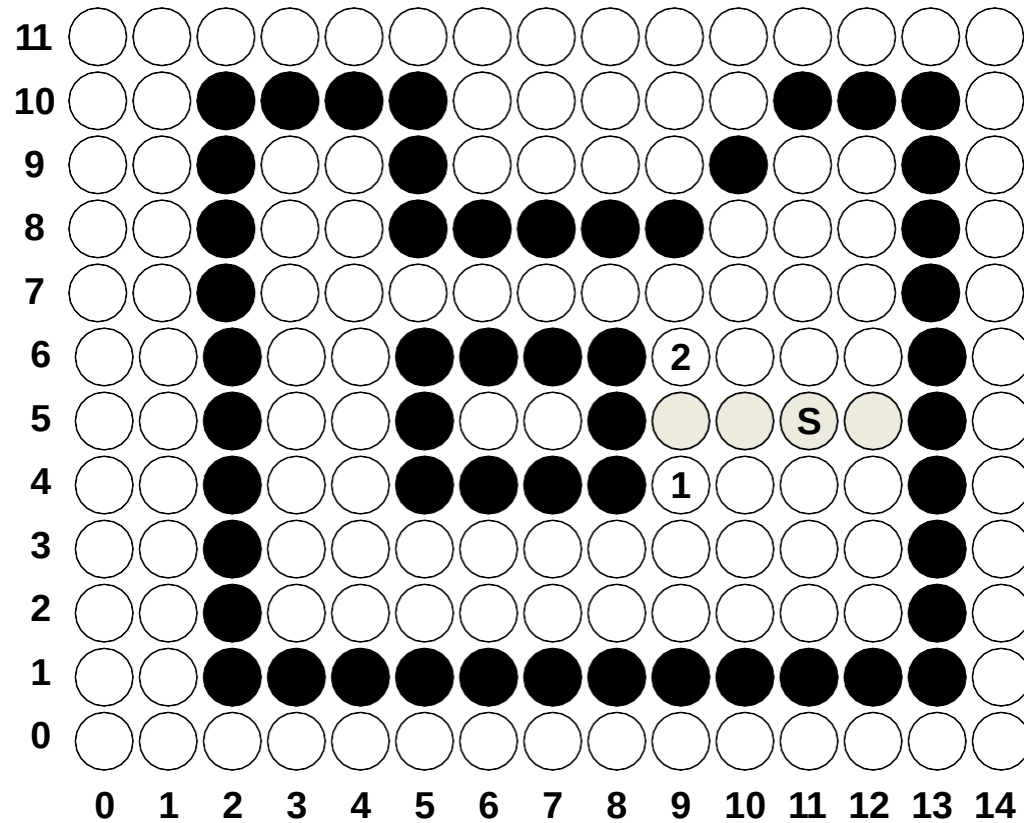
The algorithm is summarized as follows:

- **Starting** from the initial interior pixel, then fill in the contiguous span of pixels on this starting scan line.
- **Then locate** and **stack** starting positions for spans on the adjacent scan lines, where spans are defined as the contiguous horizontal string of positions bounded by pixels displayed in the area border color.
- **At each subsequent step, unstack** the next start position and repeat the process.

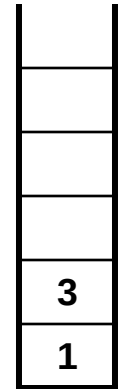
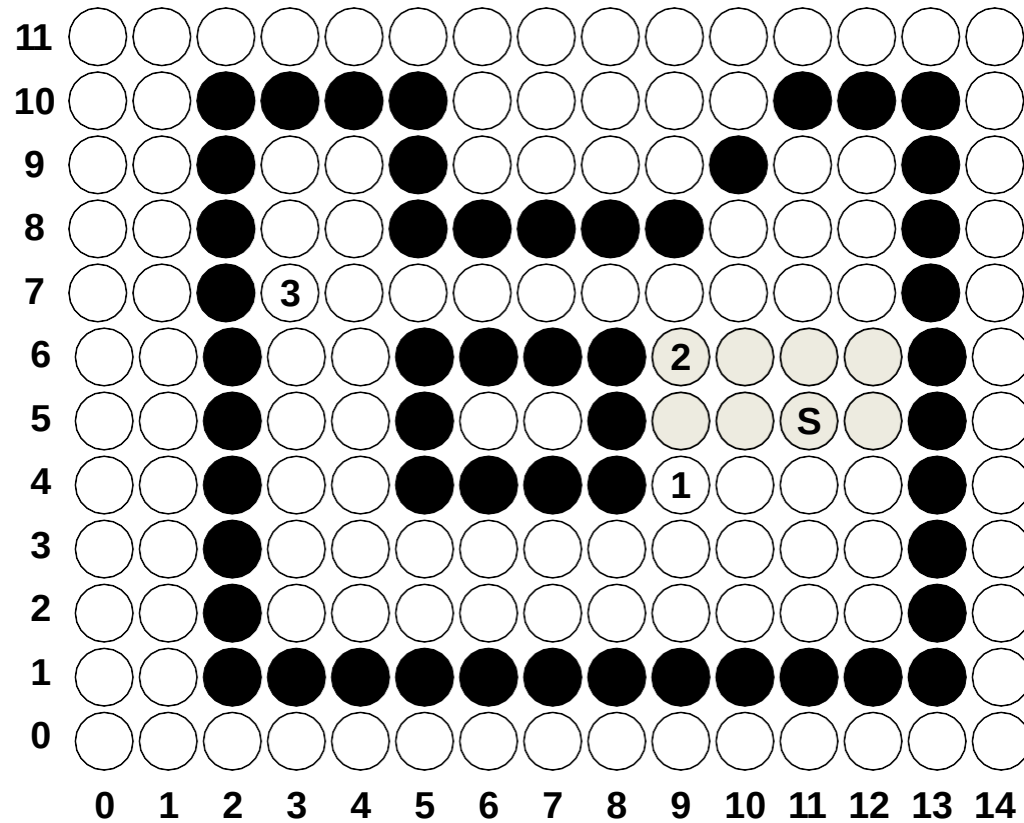
Span Flood-Fill Algorithm (example)



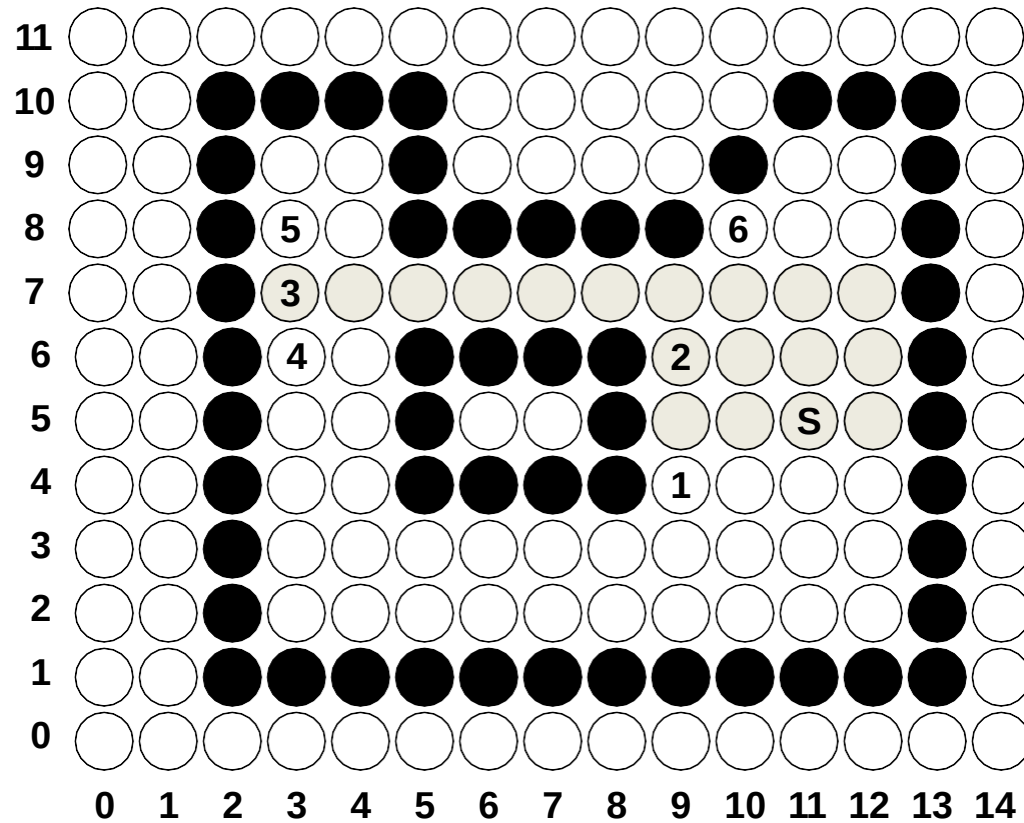
Span Flood-Fill Algorithm (example)



Span Flood-Fill Algorithm (example)

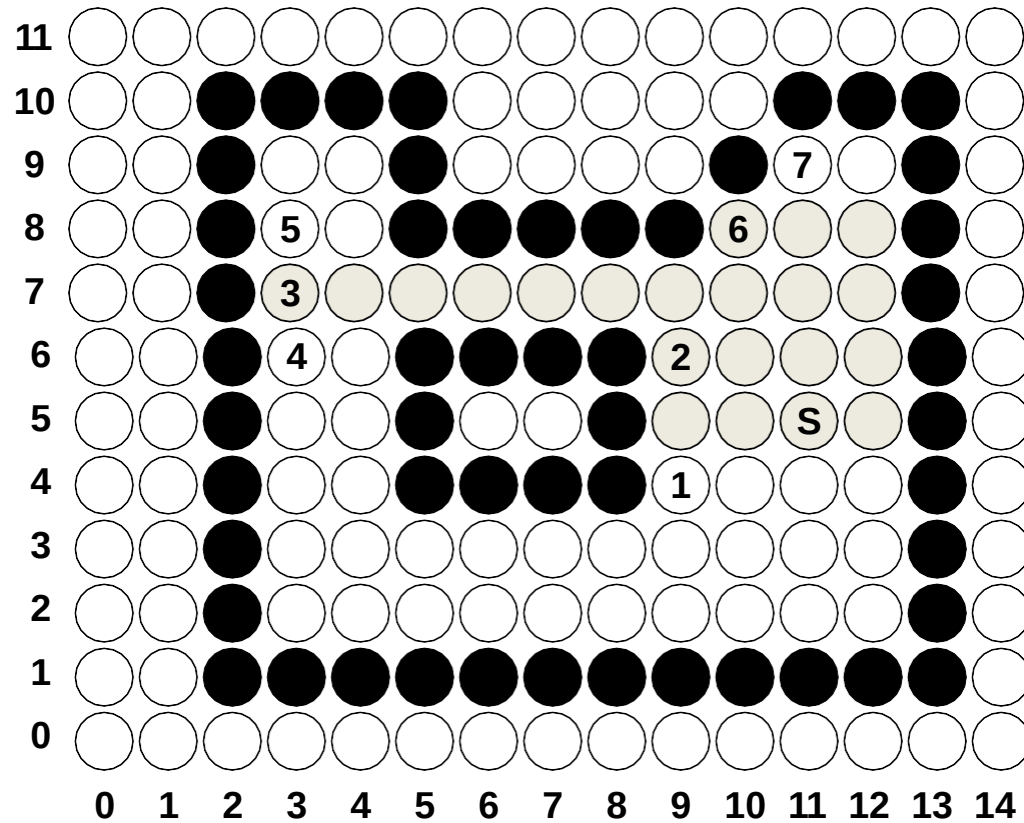


Span Flood-Fill Algorithm (example)



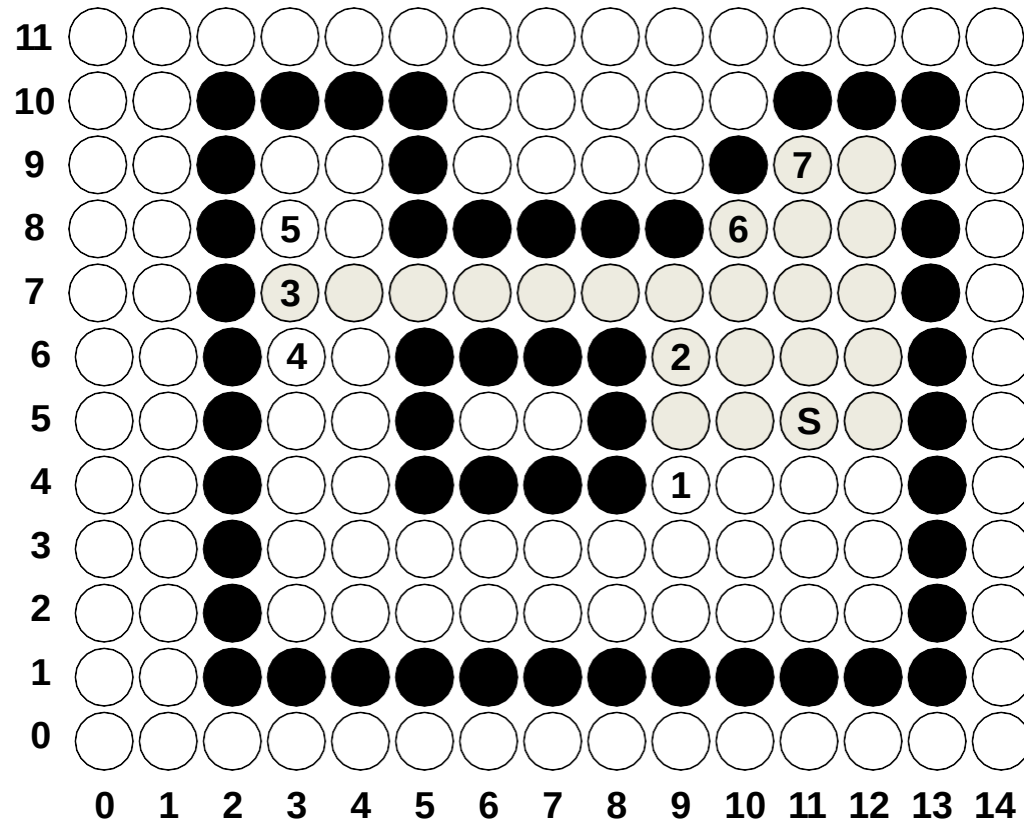
6
5
4
1

Span Flood-Fill Algorithm (example)



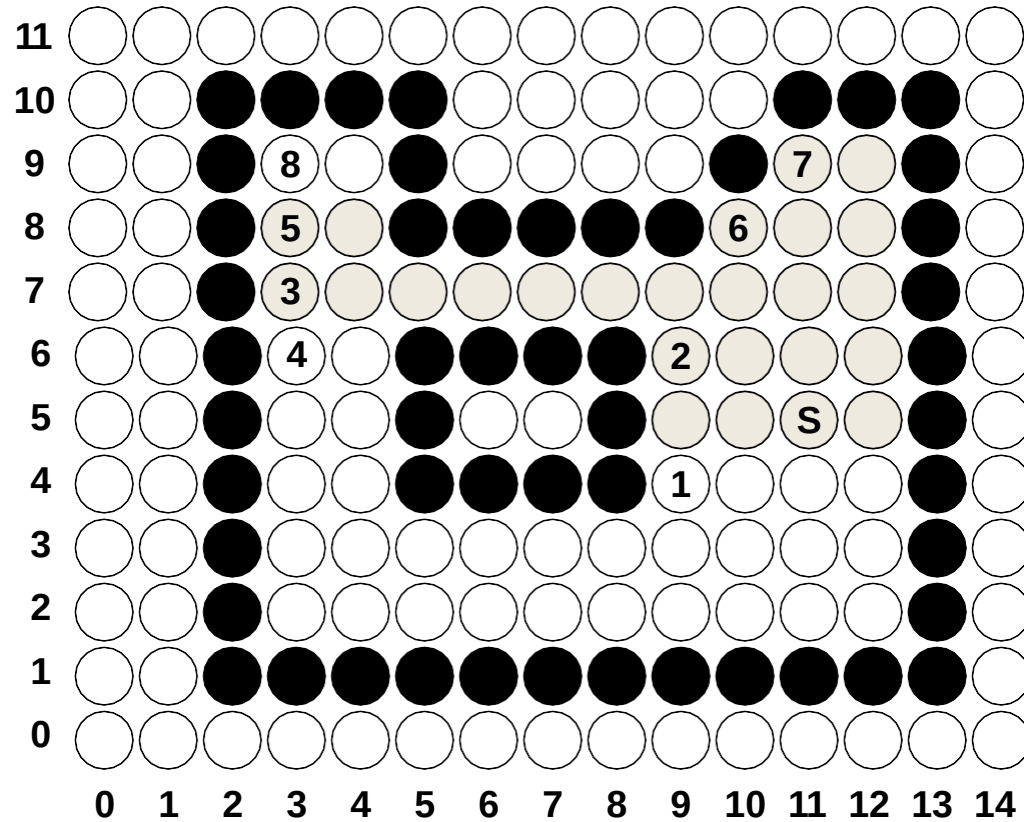
7
5
4
1

Span Flood-Fill Algorithm (example)



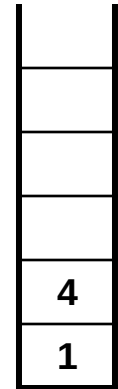
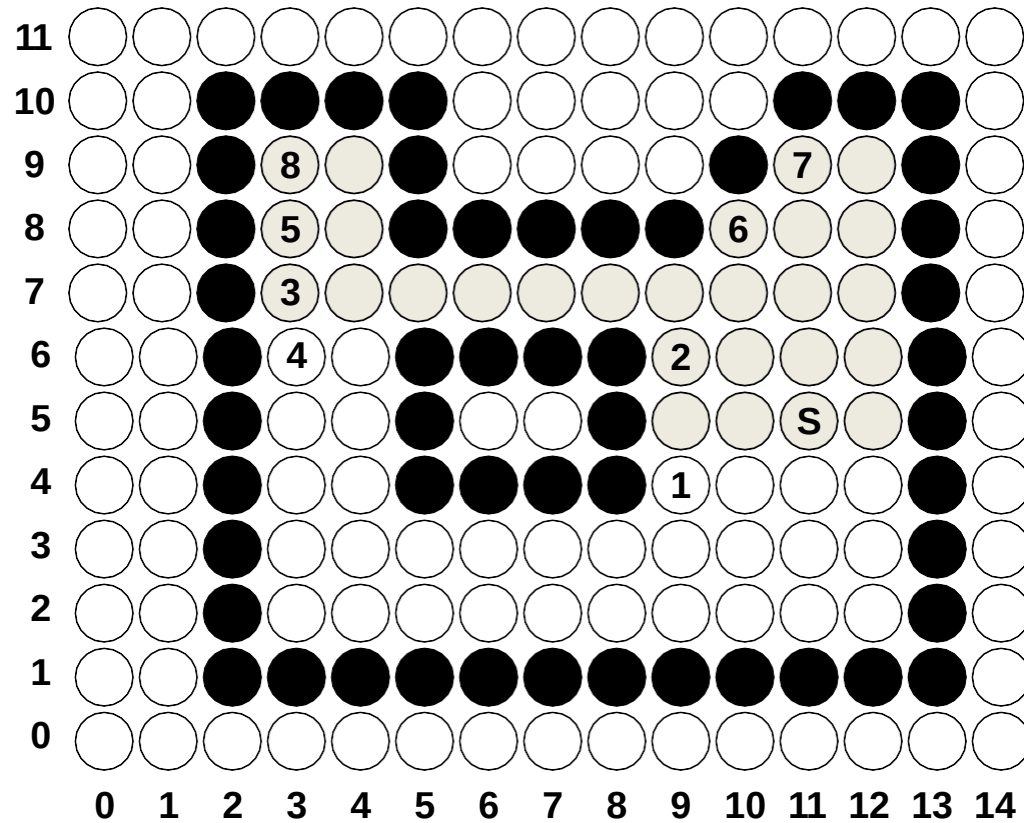
5
4
1

Span Flood-Fill Algorithm (example)

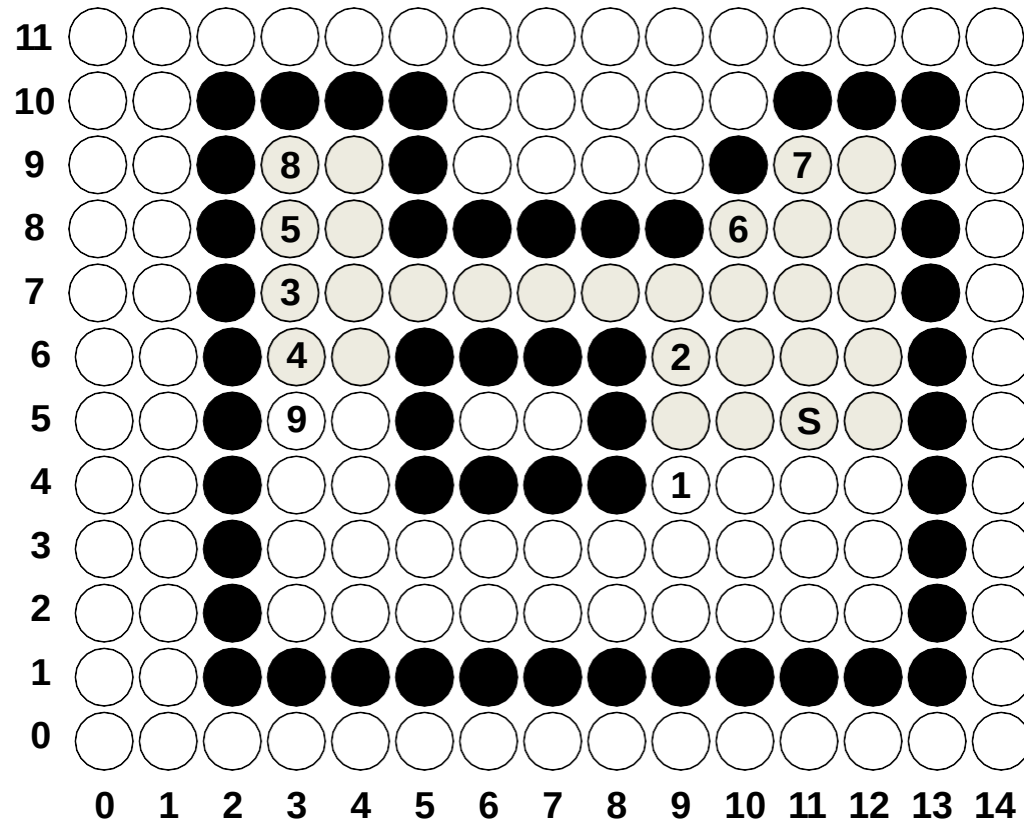


8
4
1

Span Flood-Fill Algorithm (example)

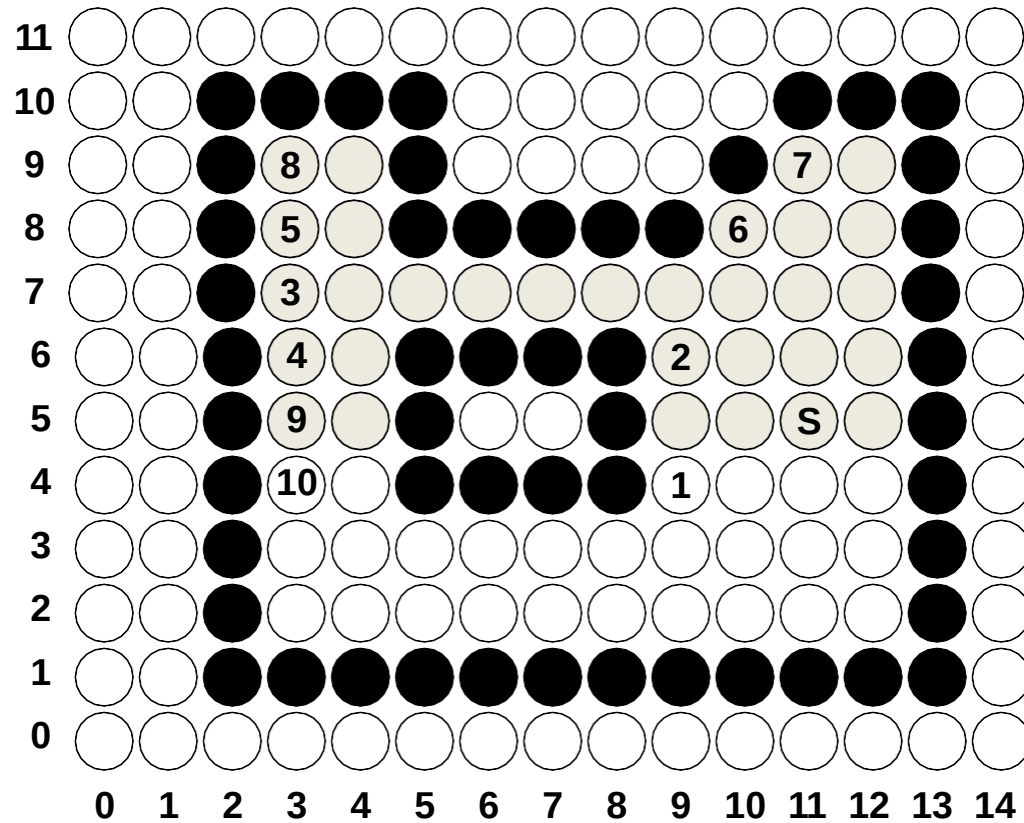


Span Flood-Fill Algorithm (example)



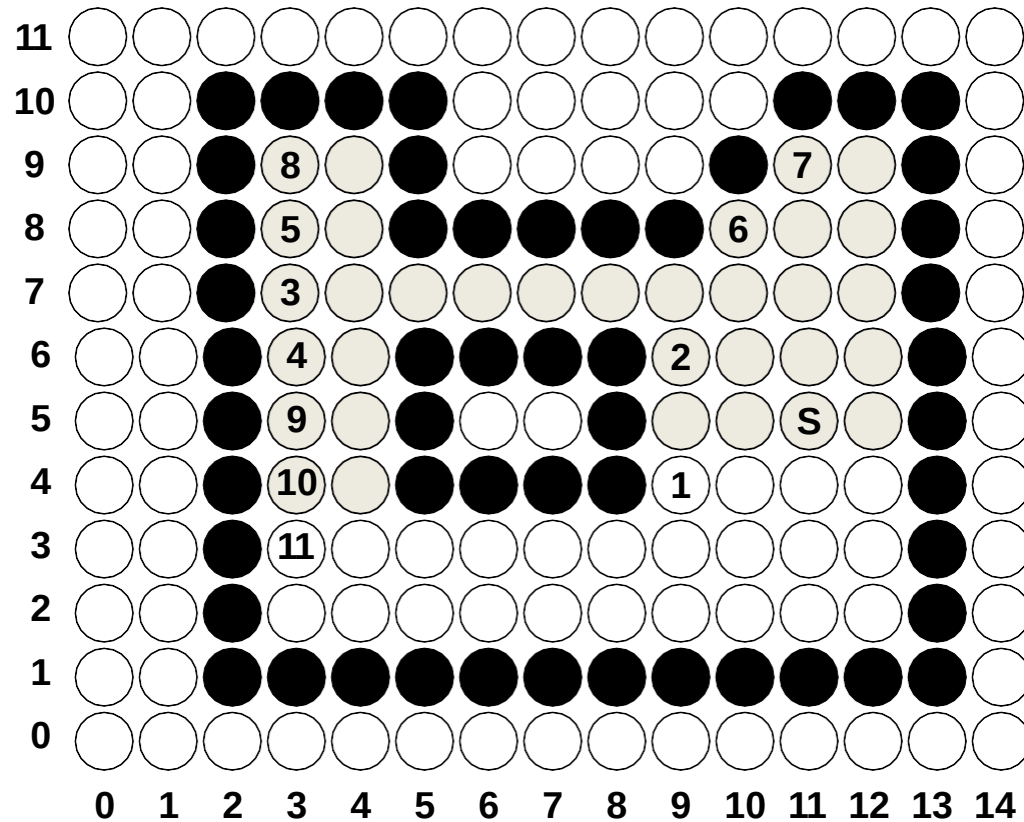
9
1

Span Flood-Fill Algorithm (example)



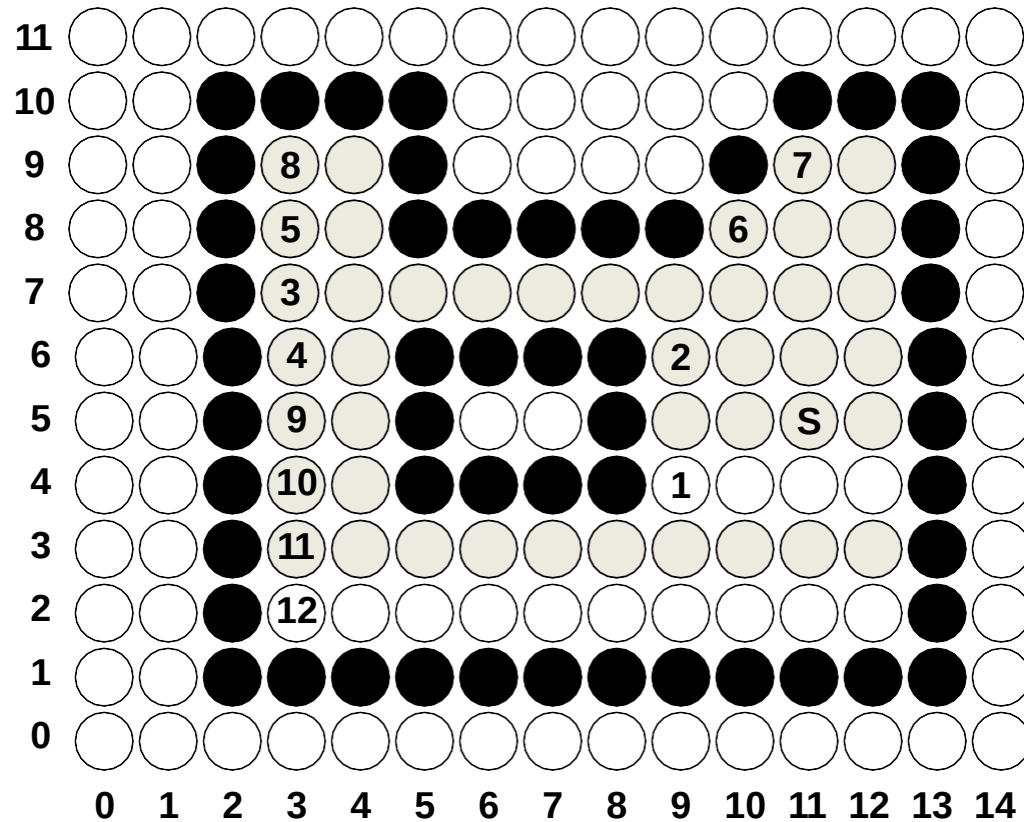
10
1

Span Flood-Fill Algorithm (example)



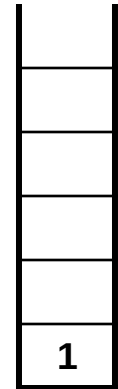
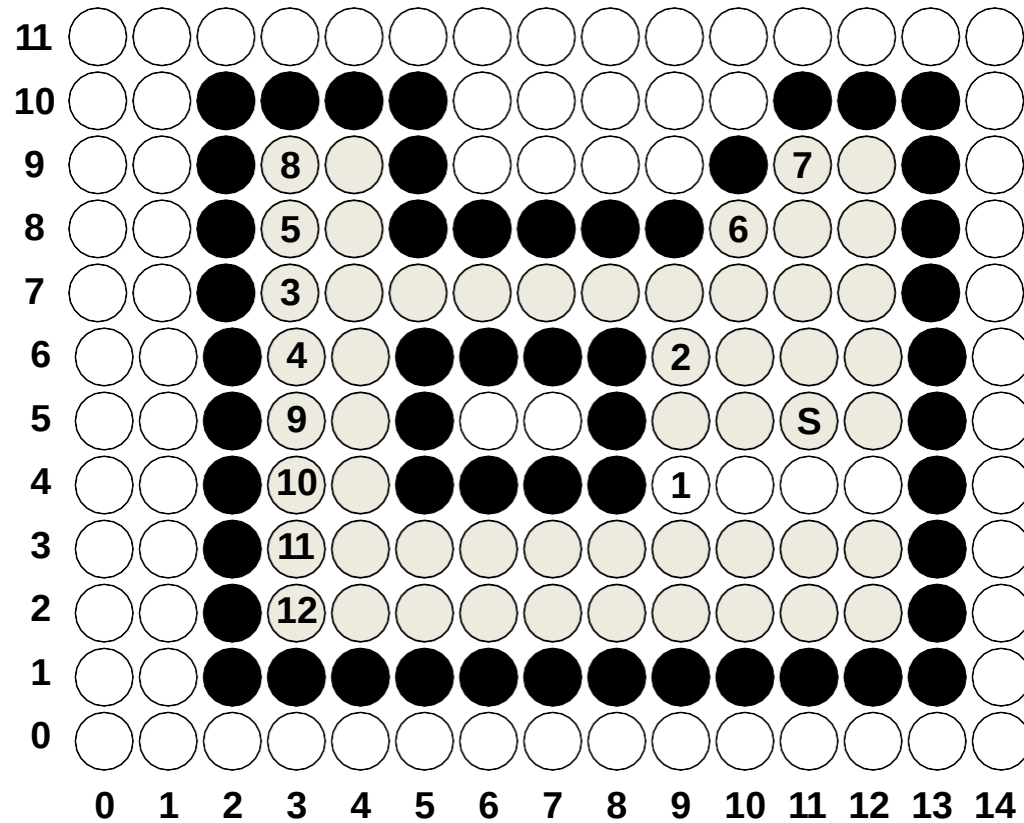
11
1

Span Flood-Fill Algorithm (example)



12
1

Span Flood-Fill Algorithm (example)



Span Flood-Fill Algorithm (example)

